LAB REPORT 01

Basic Signals, Operations & Signal Properties

Exercise 01: Continuous & Discrete-Time Signals and Complex Exponentials

> Define and plot:
> a) The continuous-time signal $x_1(t) = \cos(\pi t)$
> b) The discrete-time signal $x_2[n] = \cos(\pi n)$
> c) The real part and the imaginary part of the signal x(t) = e^(j3πt) in time of two periods.

MATLAB Implementation Code:

```
%% Continuous-time signal x1(t) = cos(pi*t)
t = linspace(-2, 2, 500); % time from -2 to 2
x1 = cos(pi*t);
figure;
plot(t, x1, 'b', 'LineWidth', 1.5);
grid on;
xlabel('t'); ylabel('x1(t)');
title('Continuous-time signal x1(t) = cos(\pi t)');

%% Discrete-time signal x2[n] = cos(pi*n)
n = -10:10; % discrete time indices
x2 = cos(pi*n);
figure;
stem(n, x2, 'r', 'filled'); % stem plot for discrete signals
grid on;
xlabel('n'); ylabel('x2[n]');
title('Discrete-time signal x2[n] = cos(\pi n)');

%% Real and Imag part of x(t) = e^{j 3\pi t}
T = 2/3; % fundamental period
t2 = linspace(0, 2*T, 500); % two periods (0 to 4/3)
x = exp(1j*3*pi*t2);

figure;
plot(t2, real(x), 'b', 'LineWidth', 1.5); hold on;
plot(t2, imag(x), '--', 'LineWidth', 1.5);
grid on;
xlabel('t'); ylabel('Value');
title('Real and Imag parts of x(t) = e^{j 3\pi t}');
legend('Re\{x(t)\} = cos(3\pi t)', 'Im\{x(t)\} = sin(3\pi t)');
```

Simulation Output Plot:

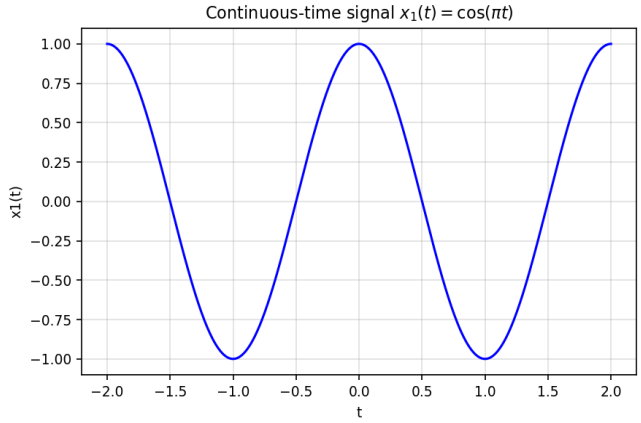


Figure 1(a): Continuous-time signal x1(t) = cos(πt)

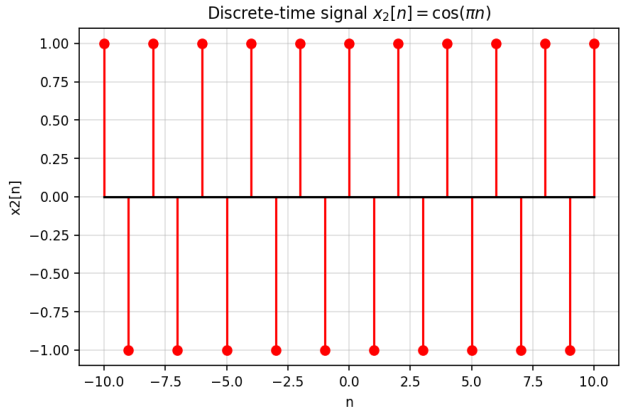


Figure 1(b): Discrete-time signal x2[n] = cos(πn)

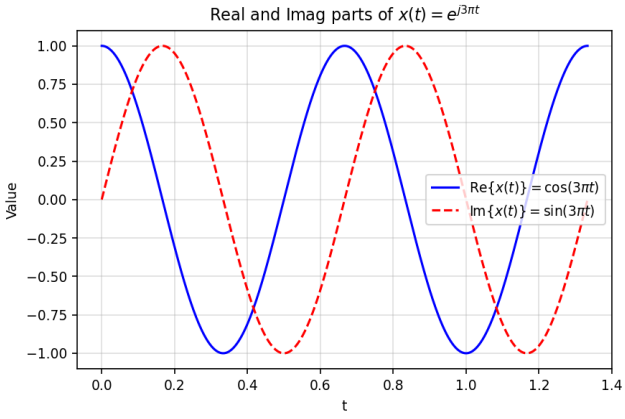


Figure 1(c): Real and imaginary parts of x(t) = e^(j3πt)

Exercise 02: Basic Elementary Signals and Combinations

> Plot in the time interval -5 ≤ t ≤ 10 the following signals:
> a) u(t), b) u(t+3), c) u(t-2) - u(t+3), d) e^(-t)u(t+2)
> e) δ(t), f) δ(t-2), g) δ(t-1) + δ(t+2)
> h) r(t), i) r(t-1), j) r(t) - r(t-2) - r(t-5) + r(t-7)
> k) $p_4(t) = \sum \delta(t - 4k)$, l) $p_2(t-1) = \sum \delta(t - 1 - 2k)$

MATLAB Implementation Code:

```
clc;
clear;
close all;

%% Time axis
t = linspace(-5, 10, 3001); % Time interval -5 <= t <= 10

%% Basic signals
u = @(x) double(x >= 0); % Unit step function
r = @(x) x .* u(x);      % Unit ramp function

% (a) u(t)
xa = u(t);

% (b) u(t+3)
xb = u(t+3);

% (c) u(t-2) - u(t+3)
xc = u(t-2) - u(t+3);

% (d) e^{-t}u(t+2)
xd = exp(-t) .* u(t+2);

%% Impulse signals
% (e) delta(t)
delta1_time = 0;

% (f) delta(t-2)
delta2_time = 2;

% (g) delta(t-1) + delta(t+2)
delta_sum_time = [-2 1];

% (h) r(t)
xh = r(t);

% (i) r(t-1)
xi = r(t-1);
```

```
% (j) r(t) - r(t-2) - r(t-5) + r(t-7)
xj = r(t) - r(t-2) - r(t-5) + r(t-7);

%% (k) p4(t) = sum delta(t-4k)
k4 = ceil(-5/4) : floor(10/4);
p4_time = 4 * k4;

%% (l) p2(t-1) = sum delta(t-1-2k)
k2 = ceil((-5-1)/2) : floor((10-1)/2);
p2_time = 1 + 2 * k2;

%% Plot all signals (Example of plotting a few subplots)
figure('Color', 'w');
subplot(4, 3, 1)
plot(t, xa, 'LineWidth', 1.5)
grid on
title('(a) u(t)')
xlabel('t')
ylabel('Amplitude')
```

Simulation Output Plot:

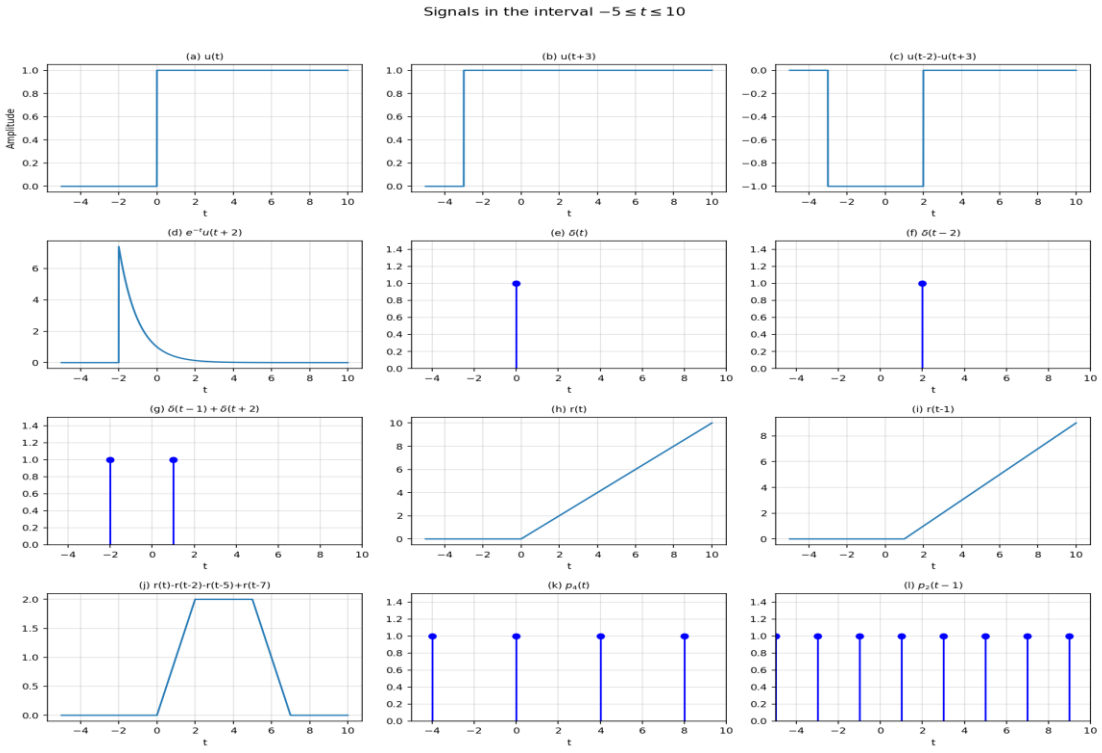


Figure 2: Elementary signals u(t), r(t), δ(t), and impulse trains over −5 ≤ t ≤ 10

Exercise 03: Even and Odd Symmetry Decomposition

> Determine if the signal x(t) = t·cos(t) is symmetric, and if yes what kind of symmetry.
> Also, decompose x(t) into its even and odd parts and confirm your analytical result via MATLAB simulation.

MATLAB Implementation Code:

```
clc;
clear;
close all;

%% Time axis
t = linspace(-6, 6, 2000);

%% Original signal x(t) = t cos(t)
x = t .* cos(t);

%% Calculate x(-t)
x_neg = (-t) .* cos(-t);

%% Even and Odd decomposition
xe = 0.5 * (x + x_neg); % Even part
xo = 0.5 * (x - x_neg); % Odd part

%% Verification
fprintf('Maximum value of even part = %.2e\n', max(abs(xe)));
fprintf('Maximum difference between x(t) and odd part = %.2e\n', max(abs(x-xo)));

%% Plotting
figure;
plot(t, x, 'k', 'LineWidth', 1.5);
hold on;
plot(t, xe, 'r--', 'LineWidth', 1.5);
plot(t, xo, 'b-.', 'LineWidth', 1.5);
grid on;
xlabel('t');
ylabel('Amplitude');
title('Even/Odd decomposition of x(t)=t cos(t)');
legend('x(t)=t cos(t)', 'Even part x_e(t)', 'Odd part x_o(t)');
```

Simulation Output Plot:

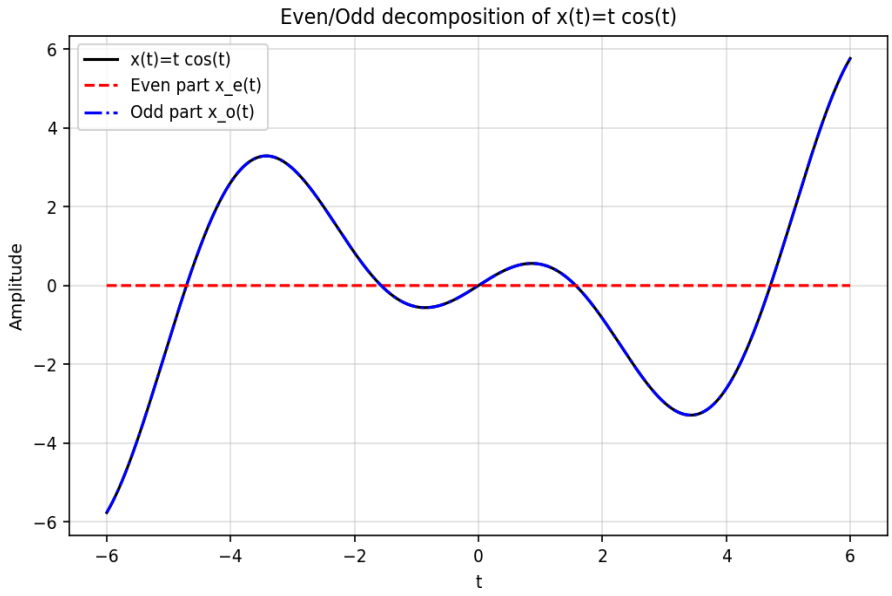


Figure 3: Even/odd decomposition of x(t) = t cos(t)

Command Window Result: Maximum value of even part = 0.00e+00,

Maximum difference between x(t) and odd part = 0.00e+00

Exercise 04: Signal Classification: Energy and Power Signals

> Determine whether the signal x(t) = t[u(t) - u(t-8)] is an energy or power signal by calculating total energy E and average power P.

MATLAB Implementation Code:

```
clc;
clear;
close all;

%% Define time axis
t = linspace(-2, 10, 5000);

%% Define unit step function
u = @(t) double(t >= 0);

%% Define the signal x(t)
x = t .* (u(t) - u(t-8));

%% Plot the signal
figure;
plot(t, x, 'LineWidth', 1.5);
grid on;
xlabel('Time t');
ylabel('x(t)');
title('Signal x(t)=t[u(t)-u(t-8)]');

%% Calculate Energy
energy_density = abs(x).^2;
E = trapz(t, energy_density);
fprintf('Energy E = %.2f\n', E);

%% Calculate Average Power
T = 1000; % Large time interval
t_power = linspace(-T, T, 100000);
x_power = t_power .* (u(t_power) - u(t_power-8));
P = (1/(2*T)) * trapz(t_power, abs(x_power).^2);
fprintf('Average Power P = %.6f\n', P);

%% Classification
if E > 0 && isfinite(E)
    fprintf('Classification: ENERGY SIGNAL\n');
elseif P > 0 && isfinite(P)
    fprintf('Classification: POWER SIGNAL\n');
else
    fprintf('Classification: Neither Energy nor Power Signal\n');
end
```

Simulation Output Plot:

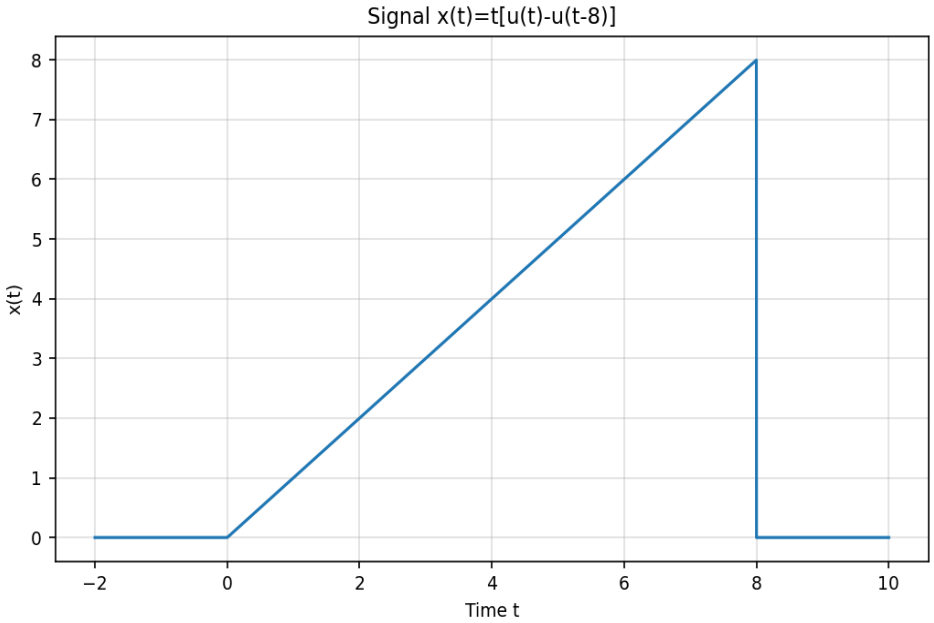


Figure 4: Signal x(t) = t[u(t) – u(t – 8)]

Command Window Result: Energy E = 170.62
Average Power P = 0.085336
Classification: ENERGY SIGNAL

Exercise 05: Time Transformations on Continuous-Time Signals

> Consider the signal x(t) = t[u(t) - u(t-5)]. Implement and plot the following transformations:
> (a) x(-t) [Time Reversal], (b) x(4t) [Compression], (c) x(0.3t) [Expansion]
> (d) x(t+2) [Time Advance], (e) x(t-3) [Time Delay], (f) x(1-3t) [Combined Scaling & Shift]

MATLAB Implementation Code:

```
clc;
clear;
close all;

%% Define unit step function
u = @(t) double(t >= 0);

%% Define original signal
% x(t) = t[u(t) - u(t-5)]
x = @(t) t .* (u(t) - u(t-5));

%% Time axis
```

```
t = linspace(-8, 10, 5000);

%% Generate transformed signals
xa = x(-t);          % (a) Time reversal
xb = x(4*t);         % (b) Compression by 4
xc = x(0.3*t);       % (c) Expansion by 1/0.3
xd = x(t+2);         % (d) Shift left by 2
xe = x(t-3);         % (e) Shift right by 3
xf = x(1-3*t);       % (f) Reflection + compression + shift

%% Plot all signals
figure('Color', 'w');

% Plot (a)
subplot(3, 2, 1)
plot(t, xa, 'LineWidth', 1.5);
grid on;
xlabel('t'); ylabel('x(-t)');
title('(a) x(-t)');

% Plot (b)
subplot(3, 2, 2)
plot(t, xb, 'LineWidth', 1.5);
grid on;
xlabel('t'); ylabel('x(4t)');
title('(b) x(4t)');

% Plot (c)
subplot(3, 2, 3)
plot(t, xc, 'LineWidth', 1.5);
grid on;
xlabel('t'); ylabel('x(0.3t)');
title('(c) x(0.3t)');

% Plot (d)
subplot(3, 2, 4)
plot(t, xd, 'LineWidth', 1.5);
grid on;
xlabel('t'); ylabel('x(t+2)');
title('(d) x(t+2)');

% Plot (e)
subplot(3, 2, 5)
plot(t, xe, 'LineWidth', 1.5);
grid on;
xlabel('t'); ylabel('x(t-3)');
title('(e) x(t-3)');

% Plot (f)
```

```
subplot(3, 2, 6)
plot(t, xf, 'LineWidth', 1.5);
grid on;
xlabel('t'); ylabel('x(1-3t)');
title('(f) x(1-3t)');

sgtitle('Time Transformations of x(t)=t[u(t)-u(t-5)]');
```

Simulation Output Plot:

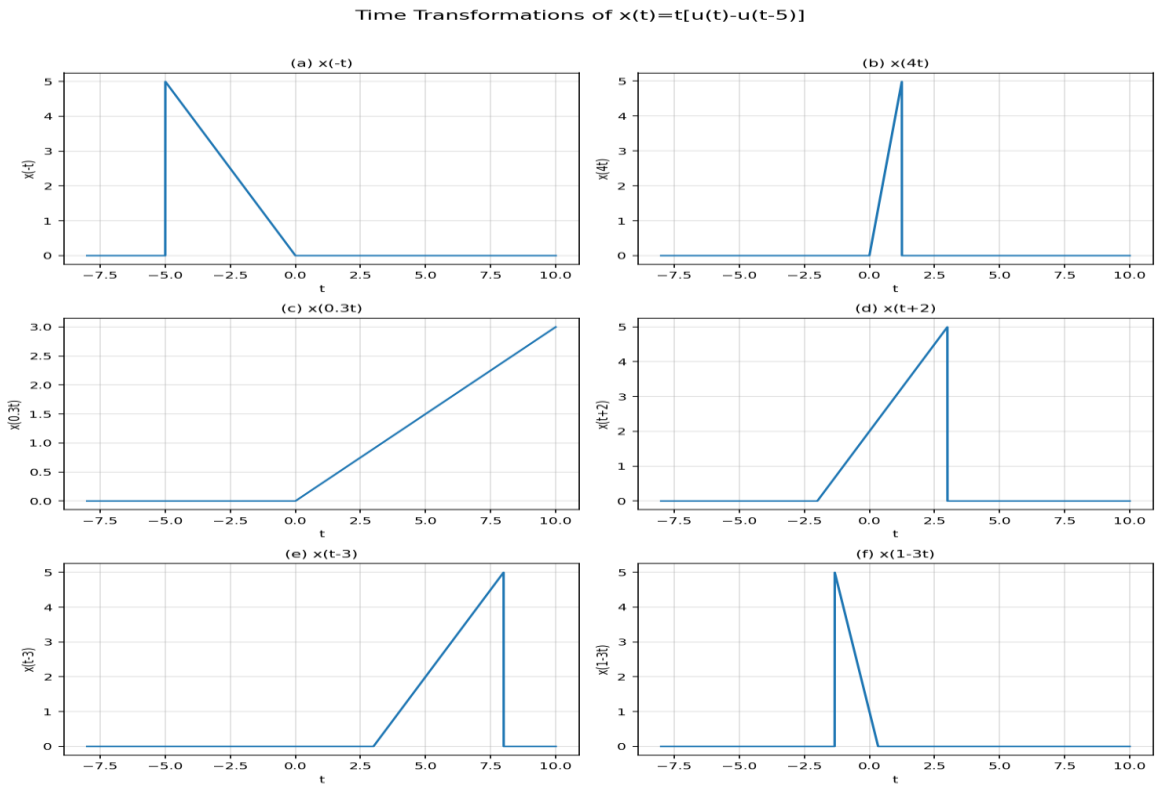


Figure 5: Time transformations of x(t) = t[u(t) − u(t − 5)]